



SISTEMAS OPERACIONAIS

AULA 6 – PROCESSOS E ESCALONAMENTO

CONTEXTUALIZANDO



Aula passada:

Processos

Aula de hoje:

Processos e Escalonamento

DEFINIÇÃO DE ESCALONADOR

- A troca de processos é feita pelo **Escalonador de Processos**

- Escalonador é o processo que escolhe qual será o próximo processo a ser executado.

- Existem diversas técnicas/algoritmos para escalonamento de processos

- Nível mais baixo do SO

MUDANÇA DE CONTEXTO

A mudança de contexto leva a um overhead de tempo (tarefa cara):

a) É preciso salvar as informações do processo que está deixando/entrando na CPU em seu BCP

b) Salvar o conteúdo dos registradores

COMPONENTES ENVOLVIDOS

Despachante → Dispatcher

Armazena e recupera o centexto.

Atualiza as informações no BCP.

Processo relativamente rápido (0,1ms).

Escalonador → Scheduler

Escolhe a próxima tarefa a receber o processador

Parte mais demorada

QUANDO O ESCALANADOR É CHAMADO

- 1) Um novo processo é criado
- 2) Quando um processo cria outro, qual executar? Pai ou filho?
- 3) Um processo chegou ao fim e um processo pronto deve ser executado
- 4) Quando um processo é bloqueado (dependência de E/S), outro deve ser executado

QUANDO E/S OCORRE, O ESCALANADOR DEVE:

- 1) Executar o processo que estava esperando esse evento,
- 2) Continuar executando o processo que já estava sendo executado, ou
- 3) Executar um terceiro processo que esteja pronto para ser executado

CATEGORIAS DO ESCALANADOR

1) Preemptivo:

- a) Quando um processo pode, por algum motivo, perder seu uso da CPU
- b) Provoca uma interrupção forçada de um processo para que outro possa usar a CPU

2) Não Preemptivo:

Permite que o processo sendo executado continue executando

CATEGORIAS DO ESCALANADOR

Condições de parada do Não Preemptivo:

- a) termine de executar
- b) solicite uma operação de entrada/saída
- c) libere explicitamente o processador, voltando à fila de prontos

ALGORITMOS DE ESCLONAMENTO

- a) Sistemas Batch (lote)
- b) Sistemas Interativos
- c) Sistemas de Tempo Real

ALGORITMOS DE ESCALONAMENTO EM BATCH

- 1) First-Come First-Served (ou FIFO)
- 2) Shortest Job First (SJF)
- 3) Shortest Remaining Time Next (SRTN)

FIRST-COME FIRST-SERVED

- 1) Não preempitivo
 - 2) Processos são executados na CPU seguindo a ordem de requisição
 - 3) Fácil de entender e programar
- Desvantagem: Ineficiente quando há processos que demoram na sua execução

FIRST-COME FIRST-SERVED

- 1) Há uma fila de prontos (3, 2, 1)
- 2) O processo 0 sendo executado na CPU
- 3) Processo só é interrompido por E/S, ou quando é finalizado

SHORTEST JOB FIRST

- 1) Não preempitivo
- 2) Deve-se prever o tempo de execução do processo
- 3) Menor processo da lista é executado primeiro
- 4) Menor turnaround (médio)

DESVANTAGENS

- a) Todos os Jobs precisam ser conhecidos de antemão
- b) Se Jobs curtos começarem a chegar, os longos podem demorar a ser executados.

SHORTEST JOB FIRST

- 1) Não preemptivo
- 2) Deve-se prever o tempo de execução do processo
- 3) Menor processo da lista é executado primeiro
- 4) Menor turnaround time (médio)

DESVANTAGENS

- a) Todos os Jobs precisam ser conhecidos de antemão
- b) Se Jobs curtos começarem a chegar, os longos podem demorar a ser executados.

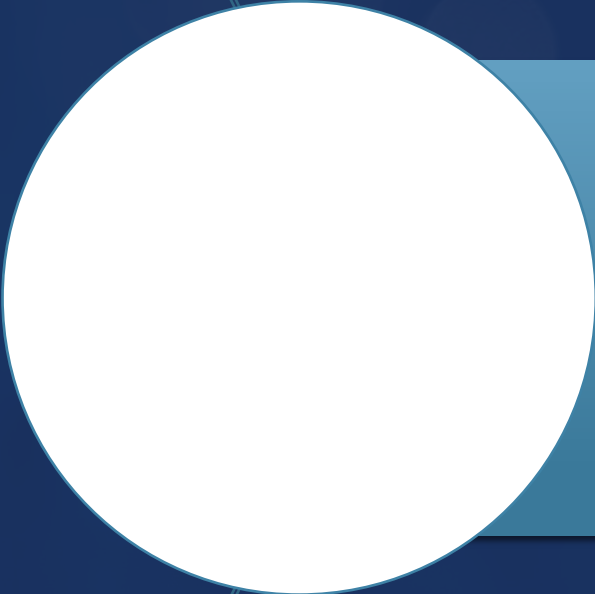
ALGORITMO SHORTEST REMAINING TIME NEXT

- 1) Preemptivo
- 2) Versão preemptiva do Shortest Job First
- 3) Processos com menor tempo de execução são executados primeiro
- 4) Se um processo novo chega e seu tempo de execução é menor do que o do processo corrente na CPU, a CPU suspende o processo corrente e executa o processo que acabou de chegar.

ALGORITMO SHORTEST REMAINING TIME NEXT

- 1) Desvantagem
- 2) Processos que consomem mais tempo podem demorar muito para serem finalizados se muitos processos pequenos chegarem (inanição ou starvation)
- 3) Exceto que, pela preempção no Shortest Remaining Time Next, o processo, mesmo rodando, seja interrompido
- 4) No Shortest Job First, se der a sorte de começar a rodar, vai até o fim

PROCESSO EM SEGUNDO PLANO - BACKGROUND



Processos com funções específicas que independem de usuários.

Exemplos:

- Recepção e envio de emails
- Serviço de impressão de arquivo

COMPONENTES ENVOLVIDOS

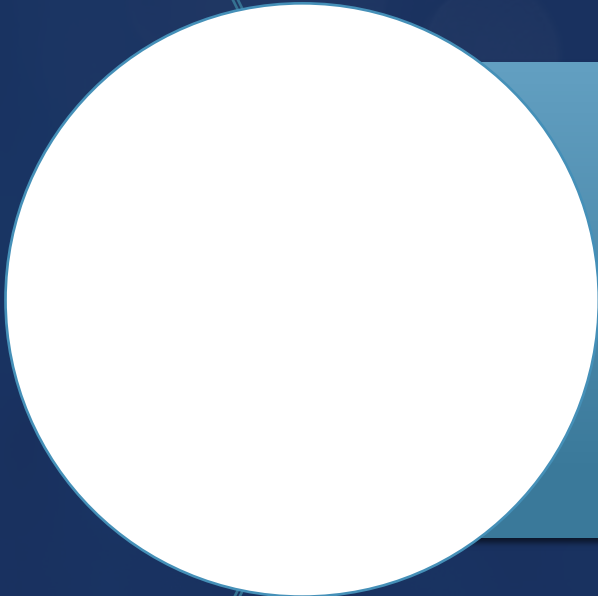
1) **Conjunto de instruções**

2) **Espaço de endereçamento** – espaço reservado para que o processo possa ler e escrever → de 0 até o máximo.

3) **Contexto de hardware** – valores nos registradores, como PC, ponteiro de pilha, e registros de propósitos gerais.

4) **Contexto de software** – atributos em geral, como lista de arquivos abertos, variáveis...

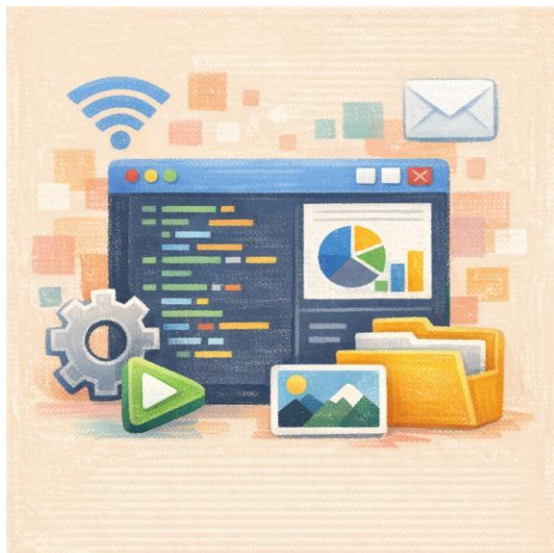
PROCESSO EM PRIMEIRO PLANO



Exemplos:

- Ler um arquivo de um pen drive
- Iniciar um programa pela linha de comando ou duplo clique.
- Navegar pela web em um browser

ESPAÇO DE ENDEREÇAMENTO



Texto → código executável do programa

Dados → as variáveis

Pilha de Execução

- Controla a execução do processo
- Empilhando chamadas a procedimentos, seus parâmetros e variáveis locais ...

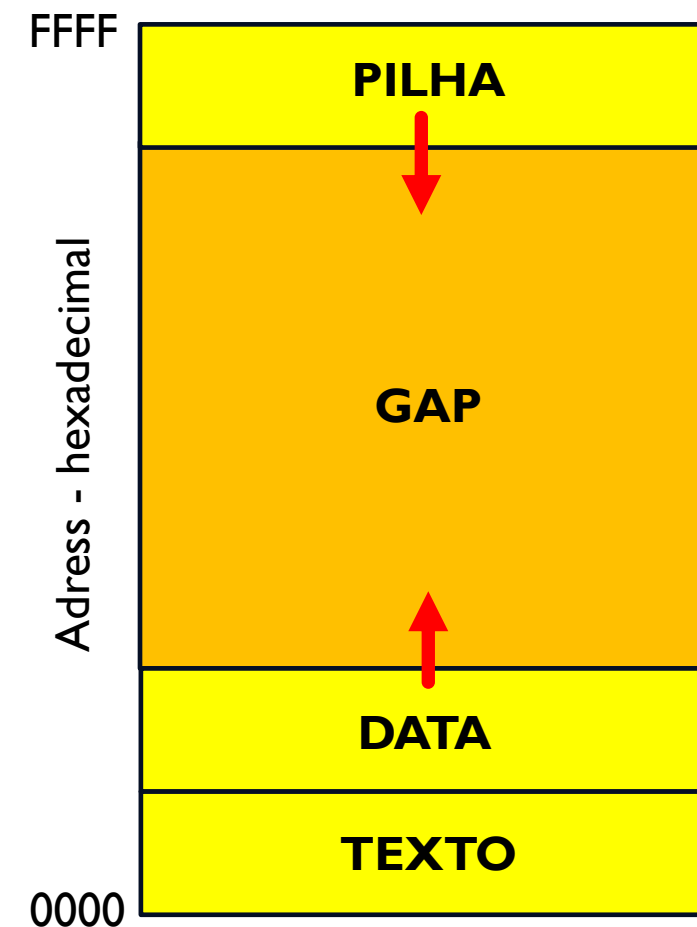


TABELA DE PROCESSOS

Também conhecida como **BCP** – Bloco Controle de Processo

Contém informações de contexto de cada processo. Ex: **ponteiros de arquivos abertos**, posição do próximo byte a ser lido em cada arquivo ...

Contém **informações necessárias para trazer o processo de volta**, caso o SO tenha que tirá-lo de execução.

Contém **estados de um processo em um determinado tempo**.

TABELA DE PROCESSOS

Tabela de Processos ou Blocos de Controle de Processos

Gerenciamento de processos

Registradores
Contador de programa
Palavra de estado do programa
Ponteiro de pilha
Estado do processo
Prioridade
Parâmetros de escalonamento
Identificador (ID) do processo
Processo pai
Grupo do processo
Sinais
Momento em que o processo iniciou
Tempo usado da CPU
Tempo de CPU do filho
Momento do próximo alarme

Gerenciamento de memória

Ponteiro para o segmento de código
Ponteiro para o segmento de dados
Ponteiro para o segmento de pilha

Gerenciamento de arquivos

Diretório-raiz
Diretório de trabalho
Descritores de arquivos
Identificador (ID) do usuário
Identificador (ID) do grupo

Campos da entrada de uma tabela de processos

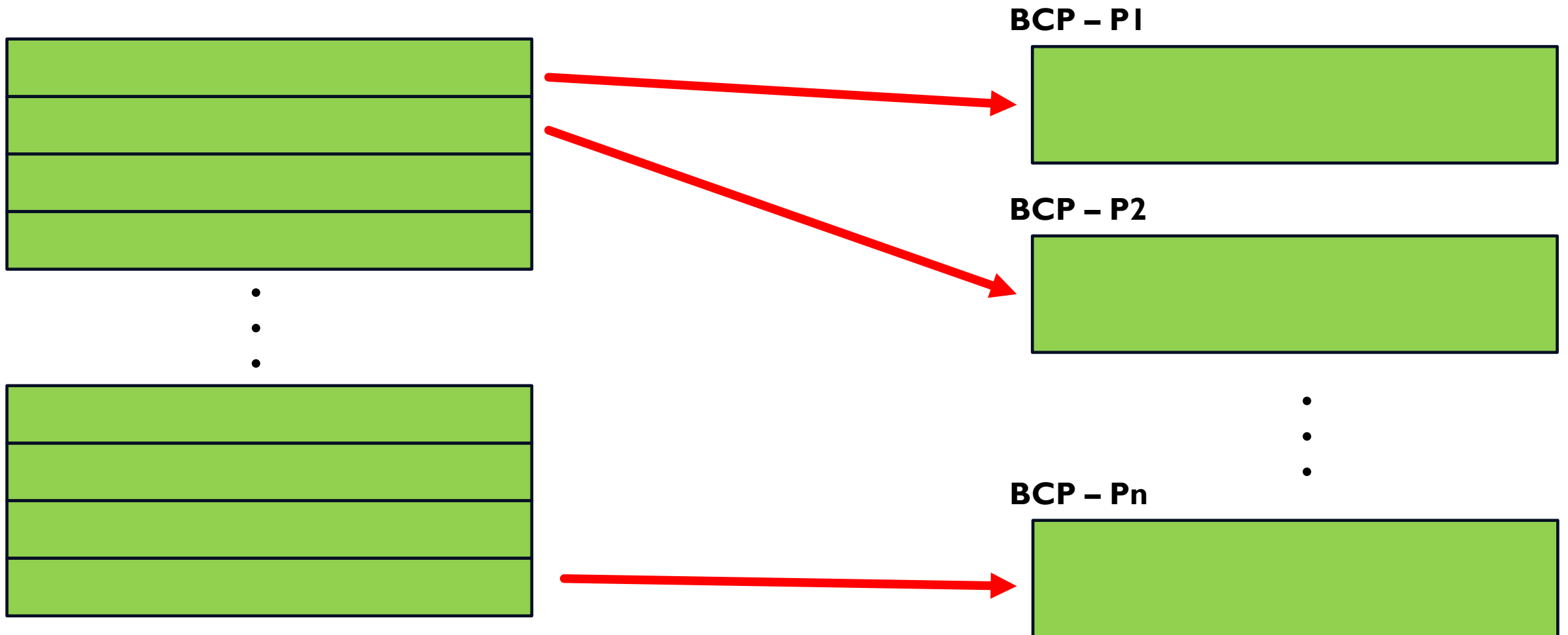
TABELA DE PROCESSOS



O **BCP** só não guarda o conteúdo do espaço de endereçamento do processo.

Assim, um processo é constituído de seu espaço de endereçamento e BCP, com seus registradores, representando uma entrada na tabela de processos.

TABELA DE PROCESSOS



PROCESSOS

Gerenciador de Tarefas

🔍 Digite um nome, editor ou PID para pesquisar

Processos

Executar nova tarefa

Nome	Status	14% CPU	36% Memória	2% Disco	0% Rede
> Microsoft Edge (38)	🛡️	0,2%	1.794,6 MB	0 MB/s	0 Mbps
> Microsoft PowerPoint		0%	226,0 MB	0 MB/s	0 Mbps
> Gerenciador WebView2 (6)	🛡️	0%	182,5 MB	0 MB/s	0 Mbps
> 🔍 Pesquisar (7)	🛡️	0%	171,1 MB	0 MB/s	0 Mbps
> Gerenciador WebView2 (6)	🛡️	0%	160,3 MB	0 MB/s	0 Mbps
> NVIDIA App (5)		0%	156,5 MB	0 MB/s	0 Mbps
Gerenciador de Janelas da Áre...		2,3%	110,3 MB	0 MB/s	0 Mbps
> Antimalware Service Executable		0%	107,9 MB	0 MB/s	0 Mbps
Secure System		0%	105,2 MB	0 MB/s	0 Mbps
> Segunda tela (4)		0%	93,5 MB	0 MB/s	0 Mbps
Windows Explorer		1,3%	79,9 MB	0 MB/s	0 Mbps
> Gerenciador de Tarefas		2,9%	78,3 MB	0 MB/s	0 Mbps
Windows Driver Foundation - ...		0%	71,1 MB	0 MB/s	0 Mbps

CRIAÇÃO DE PROCESSOS

- Inicialização do sistema
- Execução de uma chamada de sistema para criação de um processo realizada por um outro processo em execução
- Requisição de usuário para criar um novo processo – duplo clique no mouse, etc.
- Inicialização de um processo em batch
(em sistemas mainframes com processamento em batch)



PROCESSOS CRIANDO OUTROS PROCESSOS

1. No UNIX com a função `fork()`

Cria um clono do processo pai: cópias exatas na memória mas com identificadores diferentes

2. No Windows com `CreateProcess`

Cria um processo filho já carregando um novo programa nele

SYSCALL → TRAP

PROCESSOS



FINALIZANDO PROCESSOS

Término normal/voluntário

A tarefa a ser executada é finalizada.

Ao terminar, o processo executa uma chamada comunicando ao SO que terminou: `exit` no UNIX e `ExitProcess` no Win.

Término por erro

O processo sendo executado não pode ser finalizado.

Exemplo: **`gcc filename.c`**; Se o arquivo não existe.

FINALIZANDO PROCESSOS

Término com erro fatal/involuntário

Erro causado por algum bug no programa.

Ex: Divisão por zero.

Término involuntário causado por algum outro processo via chama a:

kill (unix): kill -9 9999.

TerminateProcess (Windows).

ESTADOS PROCESSOS

Executando

Realmente usando a CPU naquele momento.

Bloqueado

Incapaz de executar enquanto um evento externo não ocorrer.

Pronto

Em memória, pronto para executar ou para continuar sua execução, apenas aguardando a disponibilidade do processador.



PROFESSOR
PARISOTTO

www.parisotto.com.br